

Data Engineer Program Edición 33

5. Procesamiento en Tiempo Real con Apache Kafka y Spark
- Procesamiento en Tiempo Real con Spark y Kafka

Parte 1: Conceptos básicos

A. Docker

- ¿Qué es un contenedor y cómo se diferencia de una máquina virtual?

Un contenedor es una unidad aislada que empaqueta una aplicación junto con sus dependencias para que pueda ejecutarse de manera consistente en diferentes entornos.

La principal diferencia con una máquina virtual (VM) es que:

- ✓ Un contenedor comparte el kernel del sistema operativo anfitrión.
- ✓ Una máquina virtual incluye un sistema operativo completo dentro de la máquina virtual.
- ✓ Los contenedores son generalmente más ligeros y rápidos de iniciar.
- ✓ Las máquinas virtuales consumen más recursos porque cada una necesita su propio sistema operativo.

En nuestro ejercicio, Kafka, Zookeeper, Spark Master, Spark Worker y Jupyter se ejecutan en contenedores independientes

- ¿Qué son los volúmenes en Docker y para qué sirven?

Los volúmenes en Docker permiten almacenar datos de forma persistente y compartir archivos entre el equipo anfitrión y los contenedores.

Son importantes porque los datos almacenados solamente dentro de un contenedor pueden perderse cuando el contenedor es eliminado.

- Explica la diferencia entre docker run y docker compose.

- ✓ docker run se utiliza principalmente para crear y ejecutar un contenedor individual.
- ✓ docker compose permite definir y ejecutar varios contenedores que trabajan conjuntamente, utilizando un archivo docker-compose.yml

B. Kafka

- ¿Qué es un topic en Kafka?

Un topic es una categoría o canal lógico donde Kafka almacena y distribuye mensajes. Los productores envían mensajes a un topic y los consumidores leen mensajes de ese topic. El consumidor se suscribe a ese mismo topic para recibirlos.

- ¿Cuál es la función de Zookeeper dentro de un clúster Kafka?

Zookeeper se utiliza para gestionar información de coordinación de Kafka, como:

- ✓ Configuración del clúster.
- ✓ Estado de los brokers.
- ✓ Coordinación entre componentes.
- ✓ Información relacionada con líderes y particiones.

- Explica qué significan “particiones” y “replication factor” en un topic.

Una partición es una división del topic que permite distribuir los mensajes y procesarlos en paralelo. Esto permite aumentar el paralelismo y throughput del sistema. El replication factor indica cuántas copias de cada partición existen en diferentes brokers.

C. Spark

- ¿Qué es Spark y cuál es su propósito?

Apache Spark es un framework de procesamiento distribuido diseñado para procesar grandes volúmenes de datos de manera eficiente.

Permite realizar procesamiento:

- ✓ Batch.
- ✓ Streaming.
- ✓ Transformaciones de datos.

- ✓ Consultas SQL.
- ✓ Machine Learning.
- ✓ Procesamiento distribuido.

- Diferencia entre RDD, DataFrame y Dataset.

Los tres son estructuras utilizadas por Spark para trabajar con datos, pero tienen diferentes características

- ✓ RDD (Resilient Distributed Dataset): es la abstracción fundamental de Spark para representar una colección distribuida de objetos. Ofrece mayor control, pero requiere más trabajo del programador.
- ✓ DataFrame: representa datos organizados en filas y columnas, similar a una tabla SQL. Es la estructura que se usa habitualmente con PySpark.
- ✓ Dataset: combina las características de los RDD con las ventajas de los DataFrames y proporciona tipado fuerte.

- ¿Qué es Spark Streaming y para qué se utiliza?

Spark Streaming permite procesar datos que llegan continuamente en tiempo real o casi tiempo real.

Parte 2: Actividad práctica con Docker + Kafka + Spark

A. Se levantan los servicios y se crea topic con las siguientes instrucciones:

- ✓ `docker-compose down -v`
- ✓ `docker-compose pull`
- ✓ `docker-compose up -d`
- ✓ `docker exec -it kafka bash`
- ✓ `kafka-topics --bootstrap-server kafka:9092 --list`
- ✓ `kafka-topics \--create \--topic actividad-topic \--bootstrap-server kafka:9092 \--partitions 1 \--replication-factor 1`
- ✓ Y desde la terminal en Jupiter se ejecuta `pip install kafka-python`

```
(base) jovyan@67cafe7cf5bb:~$ pip install kafka-python
Collecting kafka-python
  Downloading kafka_python-3.0.11-py3-none-any.whl (614 kB)
    614.2/614.2 kB 1.6 MB/s eta 0:00:00
Installing collected packages: kafka-python
Successfully installed kafka-python-3.0.11
✓ (base) jovyan@67cafe7cf5bb:~$
```

B. Descripción de la solución construida con procesamiento en tiempo real

Objetivo: implementar un sistema de procesamiento de datos IoT en tiempo real utilizando Kafka y Spark Structured Streaming. Un Producer generará continuamente mediciones de temperatura, humedad y presión de diferentes sensores y las publicará en un topic de Kafka. Spark consumirá los eventos, interpretará los mensajes JSON, transformará los datos y calculará indicadores agregados por sensor. Además, identificará mediciones que superen determinados umbrales y almacenará los resultados procesados en formato Parquet.

Consideraciones

- ✓ El producer genera continuamente datos simulados entre 1 a tres segundos de temperatura, humedad y presión para diferentes sensores.
- ✓ El consumer lee los eventos, convierte los datos, detecta valores fuera de un rango establecido. La decisión de clasificar las alertas en NORMAL/ALERTA/CRÍTICO la hará Spark en el Consumer
- ✓ Para el ejercicio se tendrán **5 máquinas**
 - M001 → Sensor S001
 - M002 → Sensor S002
 - M003 → Sensor S003
 - M004 → Sensor S004
 - M005 → Sensor S005
- ✓ Cada sensor tendrá un umbral

Máquina	Sensor	Temp. máxima	Humedad máxima	Presión mínima
M001	S001	30 °C	70 %	1000 hPa
M002	S002	35 °C	75 %	1000 hPa

M003	S003	28 °C	65 %	1005 hPa
M004	S004	32 °C	80 %	995 hPa
M005	S005	30 °C	70 %	1005 hPa

- ✓ La ejecución del producer3 en la terminal es de la siguiente forma:

```
(base) jovyan@1a63e8e46bea:~$ python producer3.py
Conexión con Kafka exitosa
Iniciando productor de sensores...
Topic: actividad-topic
Enviado → M003 | T=21.21°C | H=74.87% | P=1012.67 hPa
Enviado → M005 | T=36.89°C | H=77.04% | P=1013.74 hPa
Enviado → M004 | T=33.48°C | H=84.43% | P=980.98 hPa
Enviado → M003 | T=30.0°C | H=66.3% | P=1010.67 hPa
Enviado → M005 | T=33.59°C | H=55.77% | P=992.91 hPa
Enviado → M005 | T=34.34°C | H=81.57% | P=1002.89 hPa
Enviado → M003 | T=29.29°C | H=73.71% | P=991.44 hPa
Enviado → M005 | T=35.69°C | H=74.37% | P=1013.74 hPa
Enviado → M003 | T=33.57°C | H=58.44% | P=1009.57 hPa
```

- ✓ En el consumer se ejecutan en un notebook los siguientes pasos:

- Importar librerías
- Crear SparkSession
- Definir el esquema de los sensores
- Conectar con Kafka
- Convertir el JSON recibido
- Definir los umbrales. La salida es la siguiente:

```
Umbrales definidos:
+-----+-----+-----+-----+
|maquina_id|temp_max|humedad_max|presion_min|
+-----+-----+-----+-----+
|      M001|    30.0|      70.0|    1000.0|
|      M002|    35.0|      75.0|    1000.0|
|      M003|    28.0|      65.0|    1005.0|
|      M004|    32.0|      80.0|     995.0|
|      M005|    30.0|      70.0|    1005.0|
+-----+-----+-----+-----+
```

- Cruzar sensores con sus umbrales
- Calcular las alertas
- Determinar el estado de las alertas: NORMAL / ALERTA / CRITICO
- Seleccionar el resultado final
- Preparar las rutas de salida
- iniciar el streaming. La salida es la siguiente:

```
=====
CONSUMER DE SENSORES INICIADO
=====

Kafka está esperando eventos...

>>> AHORA INICIAR EL PRODUCER <<<

Ejecutar el Producer de sensores en la terminal.
Déjar generar eventos durante unos segundos.

Los eventos aparecerán por pantalla automáticamente.
También serán almacenados en formato Parquet.

=====
```

- Mostrar las notificaciones por pantalla
- Guardar los resultados en Parquet
- verificar que los streams están activos
- Detener el consumer. La salida es la siguiente:

```
=====
STREAMING DETENIDO
=====

Los eventos recibidos han quedado almacenados en Parquet.

Ruta:
work/jupyter_storage/resultado_sensores
```

- Leer el Parquet. La salida es similar a la siguiente:

Parquet leído correctamente.

Cantidad total de registros:

22

- Mostrar las últimas 10 ejecuciones. La salida es similar a la siguiente:

```
=====
RESULTADO FINAL - ÚLTIMAS 10 EJECUCIONES
=====
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|sensor_id|maquina_id|timestamp |temperatura|humedad|presion|temp_max|humedad_max|presion_min|alerta_temperatura|alerta_humedad|alerta_presion|total_a|
|lertas|estado |tipo_alerta|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|S004 |M004 |1791072333|25.19 |74.37 |994.36 |32.0 |80.0 |995.0 |0 |0 |1 |1|
|ALERTA |PRESION |
|S002 |M002 |1791072330|38.38 |85.77 |995.87 |35.0 |75.0 |1000.0 |1 |1 |1 |3|
|CRITICO|MULTIPLE |
|S005 |M005 |1791072327|31.18 |84.13 |1002.68|30.0 |70.0 |1005.0 |1 |1 |1 |3|
|CRITICO|MULTIPLE |
|S005 |M005 |1791072325|31.81 |72.38 |991.42 |30.0 |70.0 |1005.0 |1 |1 |1 |3|
|CRITICO|MULTIPLE |
|S004 |M004 |1791072323|37.64 |76.31 |987.07 |32.0 |80.0 |995.0 |1 |0 |1 |2|
|CRITICO|MULTIPLE |
|S003 |M003 |1791072320|21.71 |76.75 |1002.71|28.0 |65.0 |1005.0 |0 |1 |1 |2|
|CRITICO|MULTIPLE |
|S002 |M002 |1791072318|29.15 |89.11 |992.63 |35.0 |75.0 |1000.0 |0 |1 |1 |2|
|CRITICO|MULTIPLE |
|S002 |M002 |1791072315|42.06 |74.46 |1006.96|35.0 |75.0 |1000.0 |1 |0 |0 |1|
|ALERTA |TEMPERATURA|
|S003 |M003 |1791072313|25.32 |71.59 |1012.57|28.0 |65.0 |1005.0 |0 |1 |0 |1|
|ALERTA |HUMEDAD |
|S005 |M005 |1791072310|26.01 |84.48 |995.95 |30.0 |70.0 |1005.0 |0 |1 |1 |2|
|CRITICO|MULTIPLE |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

- Resumen final. La salida es similar a la siguiente:

```
=====
                        RESULTADO FINAL DEL EJERCICIO
=====

Total de eventos procesados: 22

Eventos por estado:
+-----+-----+
| estado|count|
+-----+-----+
| ALERTA|   8|
| CRITICO| 10|
| NORMAL|   4|
+-----+-----+

Ubicación de los resultados Parquet:
work/jupyter_storage/resultado_sensores
```